CSC 316

Handout: Git & GitHub — Your Safety Net

What this is. How to snapshot working code, read what the agent changed, and get back when something breaks. **Companion to Lab 3 onward** — you start the habit there, and it earns its keep in Lab 4.

> **Why this matters more here than in a normal course.** In most courses git is version control. Here it is also your **undo**, because you didn't type the edit. When Claude rewrites three components and the hover-highlight you had five minutes ago stops working, ⌘Z has nothing to undo — you never made a keystroke. A commit does.

This handout covers four commands — `add`, `commit`, `diff`, `restore` — and when to reach for each.

Install Git

Check first: open a terminal and run `git --version`. If it prints a version (for example `git version 2.46.0`), Git is installed; skip to §1.

- **macOS:** running `git --version` the first time opens a prompt to install the **Command Line Developer Tools**. Click **Install** and wait a few minutes, then run `git --version` again. If no prompt appears, run `xcode-select --install`.
- **Windows:** download the installer from git-scm.com/downloads/win, run it, and accept the default options. Close and reopen PowerShell (or Command Prompt), then run `git --version`.

Then tell Git who you are, once per computer, with your own name and email:

```
git config --global user.name "Your Name"
git config --global user.email "you@mail.utoronto.ca"
```

Ask a TA if the install fails or `git --version` still is not found.

---

1. What a commit actually is

A **commit** is a labeled snapshot of every tracked file in your project, kept forever. Not a backup of one file — the whole state, at one moment, with a message on it.

Files move through three places:

```
your files          staging area          the repository
(working tree)  ──▶   git add       ──▶     git commit
```

- **`git init`** — start tracking this folder. Once per project.
- **`git add -A`** — stage everything that changed.
- **`git commit -m "…"`** — record the staged state, with a message.

> **`sv create` does not run `git init`.** It writes a `.gitignore` — so `node_modules/` and `build/` are already excluded — but the repository isn't started until you say so. That's why Activity 1 from Lab 3 onward has you run `git init` before the first commit. Skip it and `git commit` fails with *"not a git repository."*

What never goes in: `node_modules/`. It's enormous and rebuildable with `npm install`. The generated `.gitignore` already handles this; don't override it.

---

2. Commit on green, roll back don't dig

Two rules, one for each half of that heading.

Commit on green. Every time the app works, snapshot it — *before* asking for the next feature:

```
git add -A && git commit -m "brush filters the genre bars"
```

Commits are cheap and there is no such thing as too many. Each one is a labeled point you can return to.

Write the message about behavior, not files. `"brush filters the genre bars"` tells you what worked. `"updated Timeline.svelte"` tells you nothing you couldn't get from the diff. You're writing signposts for a future you who needs to find the last good state fast:

```
git log --oneline
```

```
a3f1c9e  brush filters the genre bars
7d2e440  hover a genre highlights it in the scatter
1b8c05a  four static views, no coordination
```

That log is navigable. A log of filenames is not.

Roll back, don't dig. When the next change breaks something and you can't *immediately* see why, don't spend twenty minutes debugging code you didn't write. Return to the last green state, then re-frame more precisely — *"add the brush **without changing** the hover"* — and go again.

> **The judgment call.** Debugging is worth it when you understand the code and have a hypothesis. Rolling back is worth it when you're reading unfamiliar code and guessing. With generated code you are usually in the second situation, and the roll-back is faster than your instinct says.

---

3. Reading a diff — this is the Judge step

`git diff` shows what changed since the last commit. **This is the same skill the labs train on generated code**, pointed at a different surface: the agent tells you what it did, and the diff is the receipt.

Start with the overview — which files moved at all:

```
git diff --stat
```

```
 src/lib/Timeline.svelte   |  38 ++++++++++++---
 src/lib/GenreBars.svelte  |   6 +--
 src/routes/+page.svelte   |  12 ++++--
```

Then read the detail:

```
git diff                # unstaged changes
git diff --staged       # what you're about to commit
git diff HEAD           # everything since the last commit
```

Reading the output:

```
--- a/src/routes/+page.svelte
+++ b/src/routes/+page.svelte
@@ -12,7 +12,7 @@
   let filtered = $derived(
-    yearRange ? songs.filter(d => inRange(d)) : songs
+    yearRange ? songs.filter(d => inRange(d)) : []
   );
```

`---` / `+++` name the file. `@@` marks the hunk and the line numbers. `-` is what left, `+` is what arrived. That one-character change is a real bug — `null` now means “nothing” instead of “everything,” so clearing the brush empties the dashboard.

What to look for after an agent edit:

Look for	Why it matters
Files you didn’t expect	You asked for a brush and three components changed. Ask why before accepting.
Deletions you didn’t request	The clearest sign a feature was traded away for the new one.
A quietly changed constant	A scale domain, an opacity, a default. Small diffs, large visual consequences.
`d3.select(...).append(...)`	Drift back to the classic D3 idiom (see the D3 handout §5). Re-prompt.

> **The habit.** Read the diff *before* you commit, not after something breaks. Thirty seconds on `git diff --stat` catches the “it rewrote more than I asked” case while it’s still free to undo.

---

4. Getting back

Three situations, three commands. All of them assume you committed — which is what §2 is for.

You want to…	Command	Note
Throw away all uncommitted changes and return to the last commit	`git restore .`	**Permanent.** Uncommitted work is gone. That’s the point.
Keep half-finished work but get a clean tree	`git stash` … `git stash pop`	A shelf. Stash, try something, pop it back.
Go back to a commit further back	`git log --oneline` then `git restore --source=<sha> .`	Copies that commit’s files into your working tree; you stay on your branch.

You want to…	Command	Note
See what a file looked like then, without changing anything	`git show <sha>:path/to/file`	Read-only. Safe to run any time.

> **Nothing is lost that was committed.** This is the whole bargain: commit often, and every mistake costs you minutes instead of an evening. Skip the commits and none of the commands above have anything to return to.

---

5. GitHub: an account, `gh`, and one command

Git is the tool on your machine. **GitHub** is a website that hosts copies of repositories. You need it for two things in this course: a backup that isn't your laptop, and deployment (the hosts in Deploying Your App all pull from a GitHub repo).

Get an account. Go to github.com and sign up — free, and use an email you'll keep after you graduate. Nothing else to configure.

Install `gh`, GitHub's command-line tool. It exists to remove the part that actually blocks people: authentication. Instead of generating a personal access token and pasting it into a password prompt, you run one command and click a button in your browser:

```
gh auth login          # once, ever — choose GitHub.com, HTTPS, "Login with a web browser"
```

Then one line puts your project online:

```
gh repo create csc316-lab04 --private --source=. --push
```

That creates the repository, connects your local folder to it, and pushes. After that, sending up new commits is just:

```
git push
```

> **Private or public?** Use `--private` for coursework — it keeps your solutions out of the way of next year's students. Vercel and Netlify can still deploy a private repo once you connect your GitHub account to them. Make it public later if you want it in a portfolio.

---

6. Letting Claude Code drive — and the one rule

Claude Code can run all of this. You can say *"initialize a repo and make a first commit,"* or *"show me what changed since the last commit,"* or *"push this to a new private GitHub repo,"* and it will. That's genuinely useful, and you should use it.

But there's a catch worth being deliberate about:

> **Git is your undo for when the agent breaks something. Don't let the agent be the only one holding it.** An assistant that rewrites three components and then helpfully commits them has just written over the green state you needed. The safety net and the thing it protects you from must not share a single point of failure.

The rule, which is the course's rule applied to version control:

The agent proposes. You read the diff and approve the commit.

In practice:

- **Never say "commit whatever you've done."** Say *"show me the diff"* first. Then commit — or ask for a change.
- **Write the message yourself**, or at least fix it. Agents describe files; you describe behavior (§2), and behavior is what makes the log navigable at 11pm when you need the last working version.
- **Commit before you prompt, not after.** The snapshot you want is the *working* state, and the moment you want it is before the next request — which is precisely when it's easiest to forget.

Put it in your `CLAUDE.md` once and stop repeating it:

```
Git: never commit without showing me the diff first. Never run
git restore, git reset, or git checkout without asking. I write
the commit messages.
```

That last clause matters more than it looks. `git restore .` is unrecoverable for uncommitted work, and it is exactly the sort of "helpful cleanup" an agent might run on its own initiative.

Appendix — Troubleshooting

Symptom	Likely cause & fix
`fatal: not a git repository`	You never ran `git init` in this folder — `sv create` doesn't. Run it, then `git add -A && git commit -m "baseline"`.
`git commit` says **nothing to commit**	Nothing changed, or you forgot `git add -A`. Check with `git status`.
The commit is **enormous** — thousands of files	`node_modules/` is being tracked. Confirm `.gitignore` contains `node_modules`, then `git rm -r --cached node_modules` and commit again.
`git push` asks for a **username and password**	GitHub stopped accepting passwords. Run `gh auth login` once and it stops asking.
`git push` says **no upstream branch**	The local repo isn't connected to GitHub yet. `gh repo create <name> --private --source=. --push` does the whole thing.
I ran `git restore .` and **lost work I wanted**	Uncommitted changes are unrecoverable. Committed ones aren't — `git log --oneline` and restore from a `<sha>`. This is the argument for committing on green.
Which commit was working?	`git log --oneline` and read the messages. If they're all "update files," that's the lesson — write messages about behavior.
The agent committed something **broken** over my good state	Nothing is lost: `git log --oneline`, find the last green commit, `git restore --source=<sha> .`. Then add the `CLAUDE.md` rule from §6.